Comparing Ad Effectiveness using A/B Tests

In [1]:
```
# impotring pandas and numpy

import pandas as pd
import numpy as np
```

In [2]:
```
# importing the users.csv file for analysis and reviewing the first 5 rows

users = pd.read_csv('users.csv')
users.head()
```

Out[2]:

	user_id	timestamp	device_id	clicked
0	U790620	2022-01-01 00:23:33 Saturday	M003	False
1	U584867	2022-01-01 01:30:07 Saturday	M001	False
2	U128681	2022-01-01 01:30:14 Saturday	M002	True
3	U694898	2022-01-01 01:31:55 Saturday	M003	False
4	U456823	2022-01-01 03:18:25 Saturday	M001	False

In [3]:
```
# importing the advertisements.csv for analysis and reviewing the first 5 rows

ads = pd.read_csv('advertisements.csv')
ads.head()
```

Out[3]:

	user_id	timestamp	ad_source	ad_version
0	U790620	2022-01-01 00:23:33 Saturday	Twitter	A
1	U584867	2022-01-01 01:30:07 Saturday	Google	B
2	U128681	2022-01-01 01:30:14 Saturday	TikTok	B
3	U694898	2022-01-01 01:31:55 Saturday	TikTok	A
4	U456823	2022-01-01 03:18:25 Saturday	Google	A

In [4]:
```
# importing the devices.csv for analysis and reviewing the first 5 rows

devices = pd.read_csv('devices.csv')
devices.head()
```

Out[4]:

	device_id	device_type	brand
0	M001	Mobile	Apple
1	M002	Mobile	Samsung
2	M003	Mobile	Google
3	M004	Mobile	Huawei
4	M005	Mobile	Xiaomi

In [5]:
```
# getting a count of the unique user_id's in the user_id and ads dataframes
users_user_count = users['user_id'].nunique()
ads_user_count = ads['user_id'].nunique()

print(f'Users Dataframe: {users_user_count}')
print(f'Ads Dataframe: {ads_user_count}')
```

```
Users Dataframe: 15122
Ads Dataframe: 14602
```

In [6]:
```
# merging the users and ads dataframes to make analysis easier and viewing the first 5 rows of the mer

user_ads = pd.merge(left=users, right=ads, left_on=['user_id', 'timestamp'], right_on=['user_id', 'tim
user_ads.head()
```

Out[6]:

	user_id	timestamp	device_id	clicked	ad_source	ad_version
0	U790620	2022-01-01 00:23:33 Saturday	M003	False	Twitter	A
1	U584867	2022-01-01 01:30:07 Saturday	M001	False	Google	B
2	U128681	2022-01-01 01:30:14 Saturday	M002	True	TikTok	B
3	U694898	2022-01-01 01:31:55 Saturday	M003	False	TikTok	A
4	U456823	2022-01-01 03:18:25 Saturday	M001	False	Google	A

In [7]:
```
# getting a count of how many times each ad version has been viewed

ad_view_count = user_ads.groupby('ad_version').agg({'user_id': 'count'})
# changing the column name to view_count
ad_view_count.columns = ['view_count']
# resetting the index (putting columns on the same row)
ad_view_count = ad_view_count.reset_index()
# viewing the summarized data
ad_view_count
```

Out[7]:

	ad_version	view_count
0	A	7154
1	B	7270

In [8]:
```
# getting the unique count of users who viewed the ads

ad_view_count = user_ads.groupby('ad_version').agg({'user_id': 'nunique'})
# changing the column name to view_count
ad_view_count.columns = ['unique_views']
# resetting the index (putting columns on the same row)
ad_view_count = ad_view_count.reset_index()
# viewing the summarized data
ad_view_count
```

Out[8]:

	ad_version	unique_views
0	A	7125
1	B	7232

In [9]:
```
# summarizing the data by ad source and version

social_media = user_ads.groupby(['ad_source', 'ad_version']).agg({'clicked': 'mean'})
# mulitplying the value to get an easier view of the percentages
social_media['clicked'] = round(social_media['clicked'] * 100, 2)
# naming the new column
social_media.columns = ['ctr']
# resetting the index, gettin gthe column names on the same row
social_media = social_media.reset_index()
# viewing the summarized data
social_media
```

Out[9]:

	ad_source	ad_version	ctr
0	Google	A	12.84
1	Google	B	19.88
2	Meta	A	12.97
3	Meta	B	18.80
4	TikTok	A	11.58
5	TikTok	B	20.24
6	Twitter	A	11.96
7	Twitter	B	17.51

In [10]:
```
# getting the percentage of users who clicked on each ad (CTR)

ad_ctr_pct = user_ads.groupby('ad_version').agg({'clicked': 'mean'})
# multiplying the value by 100 to get an easier view of the percentages
ad_ctr_pct['clicked'] = round(ad_ctr_pct['clicked'] * 100, 2)
# naming the new column
ad_ctr_pct.columns = ['click_rate']
# resetting the index, getting the column names on the same row
ad_ctr_pct = ad_ctr_pct.reset_index()
# viewing the summarized data
ad_ctr_pct
```

Out[10]:

	ad_version	click_rate
0	A	12.41
1	B	19.42

In [11]:
```
# creating a pivot table to view the data easier

ad_social = pd.pivot_table(user_ads,
                           index='ad_version',
                           columns='ad_source',
                           values='clicked',
                           aggfunc='mean')
# making the values easier to read and compare
ad_social = round(ad_social * 100, 2)
# changing the names of the columns to be more understandable
ad_social.columns = ['Google', 'Meta', 'TikTok', 'Twitter']
# resetting the index, getting all column headers on the same row
ad_social = ad_social.reset_index()
# viewing the summarized data
ad_social
```

Out[11]:

	ad_version	Google	Meta	TikTok	Twitter
0	A	12.84	12.97	11.58	11.96
1	B	19.88	18.80	20.24	17.51

Ad Version B is outperforming Ad Version A, especially on TikTok.

In [12]:
```
# viewing the full dataframe of the devices
devices
```

Out[12]:

	device_id	device_type	brand
0	M001	Mobile	Apple
1	M002	Mobile	Samsung
2	M003	Mobile	Google
3	M004	Mobile	Huawei
4	M005	Mobile	Xiaomi
5	M006	Mobile	vivo
6	P001	PC	Apple
7	P002	PC	Dell
8	P003	PC	HP
9	P004	PC	ASUS
10	P005	PC	Lenovo
11	P006	PC	Other
12	S001	Smartwatch	Apple
13	S002	Smartwatch	Samsung
14	S003	Smartwatch	FitBit
15	ST01	Streaming	Roku
16	ST02	Streaming	Apple TV
17	ST03	Streaming	Gaming Console
18	T001	Tablet	Apple
19	T002	Tablet	Samsung
20	T003	Tablet	Amazon
21	T004	Tablet	Microsoft

In [13]:
```
# merging the user ads dataframe with the devices dataframe on the device_id column

users_devices = pd.merge(left=user_ads, right=devices, left_on='device_id', right_on='device_id', how=
# viewing the first 5 rows of the new dataframe
users_devices.head()
```

Out[13]:

	user_id	timestamp	device_id	clicked	ad_source	ad_version	device_type	brand
0	U790620	2022-01-01 00:23:33 Saturday	M003	False	Twitter	A	Mobile	Google
1	U584867	2022-01-01 01:30:07 Saturday	M001	False	Google	B	Mobile	Apple
2	U128681	2022-01-01 01:30:14 Saturday	M002	True	TikTok	B	Mobile	Samsung
3	U694898	2022-01-01 01:31:55 Saturday	M003	False	TikTok	A	Mobile	Google
4	U456823	2022-01-01 03:18:25 Saturday	M001	False	Google	A	Mobile	Apple

In [14]:
```
# calculating the percentage of users who clicked on an ad based on their device type and ad version v

#grouped_user_devices = users_devices.groupby(['device_type', 'ad_version']).agg({'clicked':'mean'})
grouped_user_devices = pd.pivot_table(users_devices,
                                      index='ad_version',
                                      columns='device_type',
                                      values='clicked',
                                      aggfunc='mean')
# formating the values to be easier to read and compare values
grouped_user_devices = round(grouped_user_devices * 100, 2)
# renaming the column
grouped_user_devices.columns = ['Mobile', 'PC', 'Tablet']
```

```
# resetting the index, getting the column titles on the same row
grouped_user_devices = grouped_user_devices.reset_index()
# viewing the summarized data
grouped_user_devices
```

Out[14]:

	ad_version	Mobile	PC	Tablet
0	A	12.11	12.13	12.83
1	B	21.59	18.24	17.13

Ad Version B is clearly performing better than Ad Version A in this experiment, especially for Mobile.

Comparing Performance by Weekday and Weekend and Device Type

In [15]:
```
# Splitting the dataframe to create a day of week column from the timestamp

users_devices['day_of_week'] = users_devices['timestamp'].str.split(' ', expand=True)[2]
users_devices.head()
```

Out[15]:

	user_id	timestamp	device_id	clicked	ad_source	ad_version	device_type	brand	day_of_week
0	U790620	2022-01-01 00:23:33 Saturday	M003	False	Twitter	A	Mobile	Google	Saturday
1	U584867	2022-01-01 01:30:07 Saturday	M001	False	Google	B	Mobile	Apple	Saturday
2	U128681	2022-01-01 01:30:14 Saturday	M002	True	TikTok	B	Mobile	Samsung	Saturday
3	U694898	2022-01-01 01:31:55 Saturday	M003	False	TikTok	A	Mobile	Google	Saturday
4	U456823	2022-01-01 03:18:25 Saturday	M001	False	Google	A	Mobile	Apple	Saturday

In [16]:
```
# creating a boolean mask that returns True for weekend days and False if not

saturday = users_devices['day_of_week'] == 'Saturday'
sunday = users_devices['day_of_week'] == 'Sunday'

# creating a filter for weekends
is_weekend = saturday | sunday

# filtering the users devices dataframe for weekends
weekends = users_devices[is_weekend]
# viewing the first 5 rows of the weekend data
weekends.head()
```

Out[16]:

	user_id	timestamp	device_id	clicked	ad_source	ad_version	device_type	brand	day_of_week
0	U790620	2022-01-01 00:23:33 Saturday	M003	False	Twitter	A	Mobile	Google	Saturday
1	U584867	2022-01-01 01:30:07 Saturday	M001	False	Google	B	Mobile	Apple	Saturday
2	U128681	2022-01-01 01:30:14 Saturday	M002	True	TikTok	B	Mobile	Samsung	Saturday
3	U694898	2022-01-01 01:31:55 Saturday	M003	False	TikTok	A	Mobile	Google	Saturday
4	U456823	2022-01-01 03:18:25 Saturday	M001	False	Google	A	Mobile	Apple	Saturday

In [17]:
```
# getting the CTR for each day of the week by ad and device type

days_ctr = users_devices.groupby(['day_of_week','ad_version']).agg({'user_id':'nunique', 'clicked': 'm
# multiplying the value by 100 to get an easier view of the percentages
days_ctr['clicked'] = round(days_ctr['clicked'] * 100, 2)
# naming the new column
days_ctr.columns = ['unique_views', 'click_rate']
# resetting the index, getting the column names on the same row
days_ctr = days_ctr.reset_index()
# viewing the summarized data
days_ctr
```

Out[17]:

	day_of_week	ad_version	unique_views	click_rate
0	Friday	A	1060	13.85
1	Friday	B	1059	19.32
2	Monday	A	1014	12.72
3	Monday	B	1037	19.19
4	Saturday	A	1045	10.33
5	Saturday	B	994	20.78
6	Sunday	A	979	11.03
7	Sunday	B	1057	18.16
8	Thursday	A	1018	13.26
9	Thursday	B	996	19.96
10	Tuesday	A	991	12.11
11	Tuesday	B	1068	19.48
12	Wednesday	A	1046	13.48
13	Wednesday	B	1053	19.17

In [18]:
```
# getting the CTR for weekends by ad and device type
weekend_ctr = pd.pivot_table(weekends,
                             index='ad_version',
                             columns='device_type',
                             values='clicked',
                             aggfunc='mean')
# formating the values to be easier to read and compare values
weekend_ctr = round(weekend_ctr * 100, 2)
# renaming the column
weekend_ctr.columns = ['Mobile', 'PC', 'Tablet']
# resetting the index, getting the column titles on the same row
weekend_ctr = weekend_ctr.reset_index()
# viewing the summarized data
weekend_ctr
```

Out[18]:

	ad_version	Mobile	PC	Tablet
0	A	10.88	10.36	9.07
1	B	22.39	16.57	18.93

In [19]:
```
# creating a filter for weekdays

is_weekday = ~is_weekend
# filtering the users devices dataframe for weekdays
weekdays = users_devices[is_weekday]
# viewing the first 5 rows of the weekday data
weekdays.head()
```

Out[19]:

	user_id	timestamp	device_id	clicked	ad_source	ad_version	device_type	brand	day_of_week
72	U532019	2022-01-03 00:01:43 Monday	M004	False	Meta	B	Mobile	Huawei	Monday
73	U204646	2022-01-03 00:04:32 Monday	M001	False	Google	A	Mobile	Apple	Monday
74	U484678	2022-01-03 00:10:18 Monday	M003	False	Meta	A	Mobile	Google	Monday
75	U676966	2022-01-03 00:39:32 Monday	M002	True	Google	B	Mobile	Samsung	Monday
76	U145580	2022-01-03 00:45:24 Monday	M001	False	Google	A	Mobile	Apple	Monday

In [20]:

```
# getting the CTR for weekdays by ad and device type
weekday_ctr = pd.pivot_table(weekdays,
                             index='ad_version',
                             columns='device_type',
                             values='clicked',
                             aggfunc='mean')
# formating the values to be easier to read and compare values
weekday_ctr = round(weekday_ctr * 100, 2)
# renaming the column
weekday_ctr.columns = ['Mobile', 'PC', 'Tablet']
# resetting the index, getting the column titles on the same row
weekday_ctr = weekday_ctr.reset_index()
# viewing the summarized data
weekday_ctr
```

Out[20]:

	ad_version	Mobile	PC	Tablet
0	A	12.59	12.83	14.27
1	B	21.28	18.91	16.40